

Respuestas — anexadas al PDF original del Quiz 03

Esteban Mora López
Johan Adrian Torres Robles
Miguel Ángel Rodríguez Bohorquez

1. Dynamic Array from Scratch [1.0 pts]

Implementar `DynamicArray<T>` con `data`, `size`, `capacity`; constructor lanza `IllegalArgumentException` si `initialCapacity < 1`; accesos fuera de `[0, size)` lanzan `IndexOutOfBoundsException`; `removeLast` en vacío lanza excepción; `removeLast` no reduce capacidad; al llenarse, `append` duplica capacidad.

a) Implementación completa.

b) Al menos tres invariantes.

Respuesta.

I1. $0 \leq \text{size} \leq \text{capacity}$, $\text{capacity} \geq 1$.

I2. $\forall i \in [0, \text{size})$, `data[i]` contiene el elemento lógico en la posición i (el orden se preserva).

I3. $\forall i \in [\text{size}, \text{capacity})$, la celda no es alcanzable por ninguna operación pública (tras un `removeLast` queda además en `null`).

c) Complejidad worst-case.

Respuesta. `get/set`: $O(1)$ (acceso directo por índice). `removeLast`: $O(1)$ (nunca dispara `resize`). `append`: $O(1)$ si `size < capacity`, pero $\Theta(n)$ worst-case cuando dispara un `resize` (copia los n elementos existentes).

d) Por qué el worst-case de `append` no contradice el costo amortizado constante.

Respuesta. El worst-case mide la operación individual más cara aislada; el amortizado mide el promedio por operación sobre una secuencia larga. Un `append` puntual puede costar $\Theta(n)$, pero estos `resizes` ocurren con frecuencia exponencialmente decreciente (solo cuando `size` pasa por una potencia de 2 de la capacidad inicial). Con `Aggregate Method`, si ocurren r `resizes` las capacidades justo antes de cada uno son $C_0, 2C_0, 4C_0, \dots, 2^{r-1}C_0$, y el total de copias es $\sum_{j=0}^{r-1} 2^j C_0 = (2^r - 1)C_0 < 2 \cdot (2^{r-1}C_0) < 2n$, es decir $O(n)$ para n `appends` totales, dando amortizado $O(n)/n = O(1)$. No hay contradicción: son respuestas a preguntas distintas (peor caso puntual vs. promedio en una secuencia).

2. Snake: una implementación aparentemente eficiente [1.0 pts]

Representación: `DynamicArray` denso, `addHead = append`, y `REMOVE_TAIL` desplaza `data[i-1] <- data[i]` para $i = 1, \dots, \text{size} - 1$.

a) Complejidad de una llamada a `REMOVE_TAIL()`.

Respuesta. $\Theta(k)$ siempre (k = longitud actual): el bucle hace `size - 1` asignaciones, sin importar el contenido (best-case = worst-case).

b) Complejidad total de m movimientos sin comer, k constante.

Respuesta. Cada movimiento es `addHead` ($O(1)$ amortizado) + `removeTail` ($\Theta(k)$). El shifting domina: total = $\Theta(m \cdot k)$ (los m `appends` solo aportan $O(m)$).

c) “`append()` es $O(1)$ amortizado, luego `MOVE()` también” — ¿válido?

Respuesta. Falsa. El análisis amortizado de `append()` solo cubre el costo de *sus propios* `resizes`; no dice nada de `removeTail()`. Aquí `removeTail()` cuesta $\Theta(k)$ en *toda* llamada — no hay operaciones baratas que compensen las caras, luego no hay nada que amortizar en esa operación. Por lo tanto `MOVE()` = $O(1)$ amortizado + $\Theta(k)$ = $\Theta(k)$, no $O(1)$. El error es tratar “amortizado” como sinónimo de “rápido en general” en vez de analizar todas las subcomponentes de `MOVE`.

d) Bottleneck.

Respuesta. Anclar el índice lógico 0 a la posición física 0 obliga a cerrar el hueco dejado por la cola con un `shift` de $\Theta(k)$ en cada `removeTail`, independientemente de la política de capacidad — no es un problema de `resizing` sino de representación.

3. Redesign Challenge [1.0 pts]

Rediseñar Snake para $\text{addHead}() = O(1)$ amortizado y $\text{removeTail}() = O(1)$ worst-case, solo con arreglo nativo y número constante de variables auxiliares, sin delegar en el `DynamicArray` de P1.

a) Variables de estado adicionales.

Respuesta. `data` (arreglo físico, longitud `capacity`), `size` (elementos lógicos), `capacity` (longitud de `data`), y `start` = índice físico que almacena el índice lógico 0 (la cola actual) — representación de *buffer circular*.

b) Transformación índice lógico $\rightarrow$ físico.

Respuesta.

$$\text{physical}(i) = (\text{start} + i) \bmod \text{capacity}, \quad 0 \leq i < \text{size}.$$

(Con $\text{capacity} \geq 1$ y $\text{start} \in [0, \text{capacity})$, el operador módulo garantiza siempre un resultado en $[0, \text{capacity})$: la función es total y válida.)

c), d) Implementación (`addHead`, `removeTail`, `get`, `resize`) y clase completa.

Respuesta. `addHead/removeTail` nunca desplazan elementos existentes — solo escriben/leen una celda y mueven un puntero. Solo `resize` mueve elementos, recorriendo el orden lógico (usando la fórmula de b) *antes* de reescribir) y reiniciando `start=0`, lo que preserva exactamente el orden lógico sin importar la distribución física previa.

e) Invariantes.

Respuesta.

I1. $0 \leq \text{size} \leq \text{capacity}$, $\text{capacity} \geq 1$, $0 \leq \text{start} < \text{capacity}$.

I2. $\forall i \in [0, \text{size})$, `data[(start+i) mod capacity]` es el elemento lógico i (mapeo siempre válido, ver b).

I3. `addHead` y `removeTail` nunca desplazan elementos existentes; solo `resize` los mueve, exactamente una vez por llamada y en orden lógico.

f) Traza: estado inicial (cola A, cabeza B, $\text{size}=\text{capacity}=2$), luego N1(no)...N6(no).

Respuesta. Verificada por ejecución de `Main.java` (etiquetas C,D,E,F,G,H = nuevas cabezas de N1..N6 en orden):

Estado	Arreglo físico	Lógico (cola $\rightarrow$ cabeza)	start / size / capacity	Resize
Inicial	[A,B]	[A,B]	0/2/2	—
N1 (no come)	[_,B,C,_]	[B,C]	1/2/4	SI
N2 (come)	[_,B,C,D]	[B,C,D]	1/3/4	no
N3 (no come)	[E,_,C,D]	[C,D,E]	2/3/4	no
N4 (no come)	[E,F,_,D]	[D,E,F]	3/3/4	no
N5 (come)	[E,F,G,D]	[D,E,F,G]	3/4/4	no
N6 (no come)	[_,E,F,G,H,_,_,_]	[E,F,G,H]	1/4/8	SI

El primer `resize` (N1) ocurre porque el estado inicial ya llega lleno ($\text{size} = \text{capacity} = 2$): el primer movimiento sin comer dispara un `resize` antes de poder insertar la nueva cabeza transitoria.

4. Amortized Analysis: demostrar, no afirmar [1.0 pts]

Sobre la representación de P3, m movimientos, $g \leq m$ manzanas comidas, $C_0 = 2$, $C_{\text{new}} = 2C_{\text{old}}$; escribir= 1, copiar en `resize`= 1.

a) Aggregate Method: cota en función de g para el total copiado en `resizes`.

Respuesta. Por el orden `addHead` $\rightarrow$ `removeTail`, el tamaño estable k sube en 1 por manzana y nunca baja. Tras g manzanas, $k_{\text{max}} = 2 + g$; el tamaño transitorio máximo (justo tras un `addHead`, antes del `removeTail`) es $L_{\text{max}} = k_{\text{max}} + 1 = g + 3 = \Theta(g)$. Cada `resize` copia exactamente la capacidad vigente en ese instante; con capacidades $2, 4, 8, \dots, 2^r$ antes de cada uno de los r `resizes`, el total de copias es $\sum_{j=1}^r 2^j = 2^{r+1} - 2 < 4L_{\text{max}} = O(L_{\text{max}}) = O(g)$ (pues $2^r < L_{\text{max}} \leq 2^{r+1}$). **Total copiado** = $O(g)$.

b) Por qué la geométrica es lineal (no basta $1 + 2 + 4 + \dots = O(n)$).

Respuesta. Sea n el número final de elementos. El j -ésimo `resize` ocurre *exactamente* cuando $\text{size} == \text{capacity}$; en ese instante la capacidad vigente es $C_j = 2^j$ (duplica desde $C_0 = 2$), y por I1/I2 hay *exactamente* C_j elementos almacenados — el término 2^j de la suma es literalmente la capacidad y el número de copias de ese `resize`, no una coincidencia notacional. Como el `resize` r solo ocurre si $n > 2^{r-1}$, se tiene $2^r < 2n$, luego $\sum_{j=1}^r C_j = 2^{r+1} - 2 < 2 \cdot 2^r < 4n = O(n)$.

c) Costo total de los m movimientos y amortizado de `addHead()`.

Respuesta. Cada movimiento ejecuta exactamente un `addHead` (escritura= 1) más, ocasionalmente, un `resize`. Costo total = $m \cdot 1 + O(g) = O(m)$ (pues $g \leq m$). Amortizado por `addHead` = $O(m)/m = O(1)$.

d) Roles distintos de m y g .

Respuesta. m determina el número de operaciones y el costo “constante por movimiento” (las m escrituras de nueva cabeza, total m). g determina cuánto crece la estructura y por tanto la magnitud de los `resizes` (parte a–b depende de g , no de m): con m grande pero $g = 0$, no se dispara ningún `resize` adicional tras el transitorio inicial. Costo de escritura $\rightarrow$ depende de m ; costo de copia por `resize` $\rightarrow$ depende de g (vía $L_{\max} = g + 3$).

e) Potential Method (créditos suficientes pese a `removeTail`).

Respuesta. Sea $\Phi = 2 \cdot \text{size} - \text{capacity}$. `addHead` sin `resize`: $\text{real} = 1$, $\Delta\Phi = 2$, amortizado = $3 = O(1)$. `addHead` con `resize` ($\text{size} = C \rightarrow \text{capacity} = 2C$, copiando C): $\text{real} = C + 1$, $\Delta\Phi = 2 - C$, amortizado = $3 = O(1)$. `removeTail`: $\text{real} = 1$, $\Delta\Phi = -2$, amortizado = -1 (devuelve crédito). **Por qué $\Phi \geq 0$ siempre:** sea k_{prev} el tamaño estable en el último `resize` (capacidad fijada en $C = 2k_{\text{prev}}$). Como el tamaño estable k de esta serpiente es *no-decreciente* (parte a), en cualquier instante posterior $k \geq k_{\text{prev}}$, luego $\Phi = 2k - C = 2(k - k_{\text{prev}}) \geq 0$ fuera del transitorio, y $\Phi = 2(k + 1) - C \geq 2 > 0$ durante el transitorio. Como $\Phi \geq 0$ siempre (y $\Phi_0 = 2$ es $O(1)$), la cota $\sum \text{real} \leq \sum \text{amortizado} + \Phi_0 - \Phi_{\text{final}} \leq \sum \text{amortizado} + \Phi_0$ es válida: el crédito nunca se agota, incluso con `removeTail` devolviéndolo.

5. The Adversarial Snake [1.0 pts]

Políticas al llenarse: A: $C + 1$; B: $2C$; C: $[1, 5C]$. Longitud máxima transitoria n .

a) El orden de crecimiento no afecta el total copiado (sin contracción); ¿qué necesitaría el adversario?

Respuesta. Sin contracción, cada `resize` ocurre exactamente cuando el arreglo se llena; la secuencia de capacidades visitadas $C_0 \rightarrow C_1 \rightarrow \dots$ es determinista dada la política, y solo depende de *cuántas* inserciones netas ocurren ($= n$), no de *cuándo* dentro de la secuencia de movimientos. El adversario necesitaría poder **contraer** la estructura (reducir capacidad al bajar `size`) para forzar que un recrecimiento posterior vuelva a cruzar umbrales ya pagados.

b–e) Cuadro comparativo (justificación).

Respuesta. Política $C+1$: capacidades $2, 3, \dots, n \Rightarrow \Theta(n)$ `resizes`; copias $\sum_{C=2}^{n-1} C = \Theta(n^2)$; amortizado $\Theta(n^2)/n = \Theta(n)$ (no constante); desperdicio tras `resize` = 0 (llena exactamente). Política $2C$: $\Theta(\log_2 n)$ `resizes`; copias $\Theta(n)$ (parte 4b); amortizado $\Theta(1)$; tras `resize`, $\text{size} = C_{\text{old}} + 1$ con capacidad $2C_{\text{old}} \Rightarrow$ desperdicio $\approx C_{\text{old}} \approx C_{\text{new}}/2$ ($\leq 50\%$). Política $[1, 5C]$: $\Theta(\log_{1.5} n) = \Theta(\log n)$ `resizes`; copias $\Theta(n)$ (mismo argumento con razón 1,5); amortizado $\Theta(1)$; desperdicio $\approx 0,5C_{\text{old}} \approx C_{\text{new}}/3$ ($\approx 33\%$).

Política	# Resizes	Total copias	Amortizado/crecim.	Espacio sin usar tras <code>resize</code>
$C + 1$	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$	0 (0 %)
$2C$	$\Theta(\log n)$	$\Theta(n)$ ($\approx 2n$)	$\Theta(1)$	$\approx C_{\text{old}}$ ($\approx 50\%$)
$[1, 5C]$	$\Theta(\log n)$	$\Theta(n)$ ($\approx 3n$)	$\Theta(1)$	$\approx 0,5C_{\text{old}}$ ($\approx 33\%$)

f) Trade-off y familia general $C_{\text{new}} = \lceil \alpha C_{\text{old}} \rceil$.

Respuesta. $C+1$ minimiza memoria desperdiciada (0 %) pero es $\Theta(n)$ por inserción — inútil en tiempo. $2C$ y $[1, 5C]$ logran $O(1)$ amortizado a costa de reservar memoria de más justo tras un `resize`; factores menores hacen más `resizes` y desperdician menos memoria, a cambio de una constante mayor en el total de copias.

No es necesario $\alpha = 2$: para cualquier $\alpha > 1$ constante, las capacidades en los sucesivos `resizes` forman (salvo el techo, que solo añade $O(1)$ por término) una geométrica de razón α hasta superar n , con $r \approx \log_{\alpha}(n/C_0)$ términos. El total de copias es

$$\sum_{j=0}^r C_0 \alpha^j = C_0 \frac{\alpha^{r+1} - 1}{\alpha - 1} < \frac{\alpha}{\alpha - 1} (C_0 \alpha^r) \approx \frac{\alpha}{\alpha - 1} n = O(n),$$

y como $\frac{\alpha}{\alpha-1}$ es constante (no depende de n) para todo $\alpha > 1$ fijo, el costo amortizado por inserción es $O(1)$ para *cualquier* $\alpha > 1$, no solo $\alpha = 2$. El valor de α solo cambia la constante multiplicativa (crece sin cota cuando $\alpha \rightarrow 1^+$) y la fracción de memoria desperdiciada ($\rightarrow 1 - 1/\alpha$). Para $\alpha \leq 1$ la capacidad no crece: no es una política válida. Total de copias en función de α y n : $\Theta(\frac{\alpha}{\alpha-1}n)$, para todo $\alpha > 1$ constante.